



ICT306 Advanced Cybersecurity

WEEK 6 Authentication

Exploring the foundations of user authentication — from passwords and tokens to biometrics and remote verification protocols.

Resources

Prescribed Reading

Chapter 3 — Computer Security:
Principles and Practice

William Stallings and Lawrie Brown

Recommended Reading

Whitman, Michael E., and Herbert J.
Mattord. *Principles of Information
Security*. Cengage Learning, 2021.

Module 9



User Authentication

User authentication is a **fundamental security building block** — it forms the basis of access control and user accountability across all secure systems. It is the process of verifying an identity claimed by or for a system entity.

Step 1: Identification

The user presents an identifier (e.g., username or user ID) to the system.

Step 2: Verification

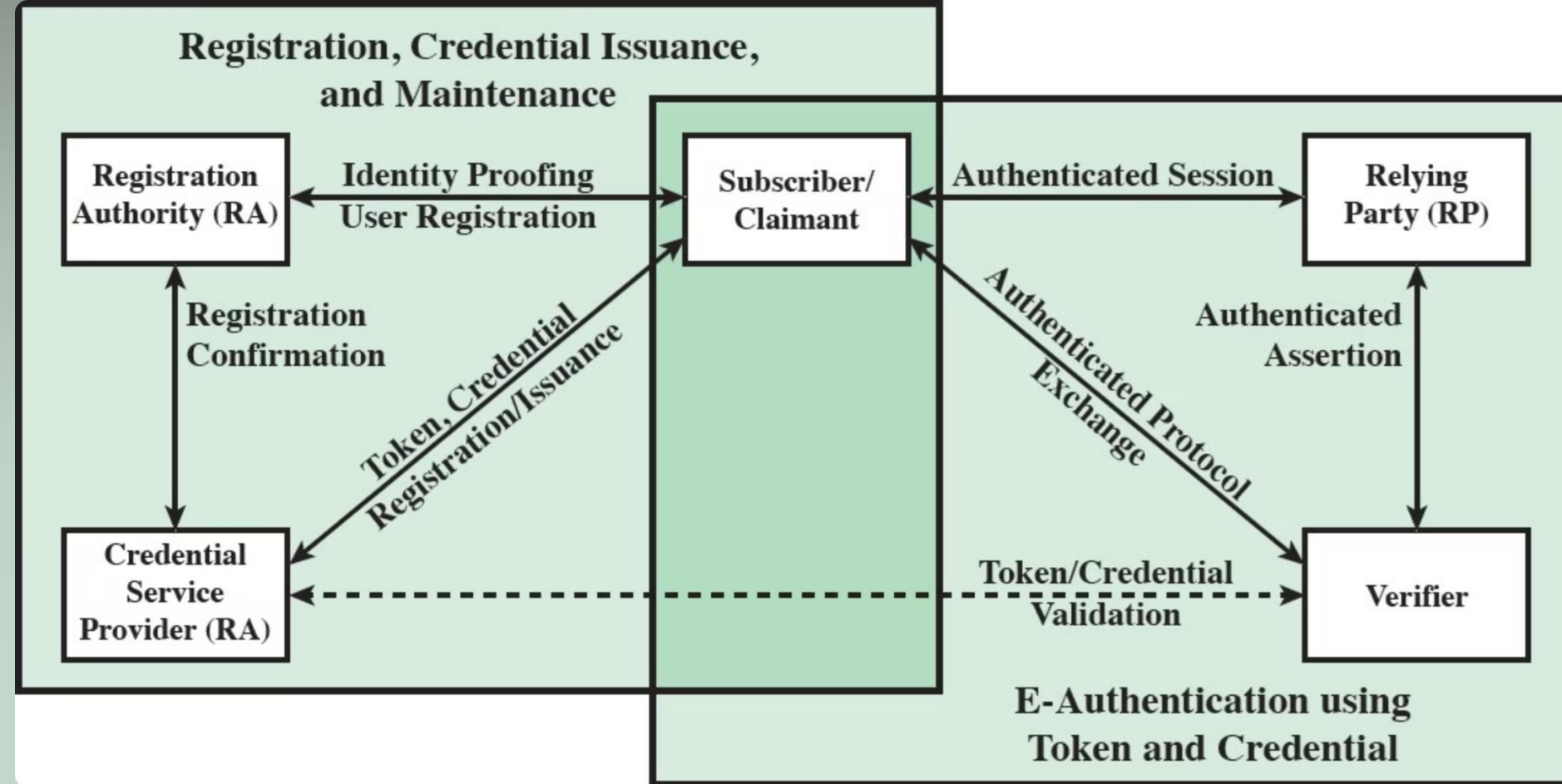
The system binds the entity (person) to that identifier using a credential or proof.

- ❏ User authentication is **distinct from message authentication**, which verifies the integrity and origin of a message rather than confirming a user's identity.



A Model for Electronic User Authentication

NIST SP 800-63-2 defines the key components and flows involved in electronic user authentication, including registration, credential issuance, and the authenticated session exchange between the claimant and relying party.



Types of User Authentication

- There are **four fundamental means** of authenticating a user's identity.
- Each can be used independently or combined for stronger, multi-factor authentication.
- All carry inherent trade-offs between security, usability, and cost.



Something You Know

Passwords, PINs, security questions — knowledge-based credentials.



Something You Are

Fingerprint, retina, iris — static biometric characteristics.



Something You Possess

Physical keys, tokens, smartcards — possession-based credentials.



Something You Do

Voice pattern, handwriting, gait — dynamic biometric behaviors.

Password-Based Authentication

- **Password-based authentication** is the most widely deployed user authentication method.
- The user **provides** a name/login and a password, and the system compares the submitted password against the stored value for that login.

→ Identity Verification

Confirms the ID of the user logging into the system.

→ Authorization Check

Determines whether the user is authorized to access the system or service.

→ Privilege Assignment

Establishes the user's privilege level and is used in discretionary access control decisions.



Password Vulnerabilities & Countermeasures (I)

- **Passwords** face a wide range of attack vectors.
- Understanding each threat and its corresponding countermeasure is essential for designing robust authentication systems.

Offline Dictionary Attack

- **Prevent unauthorized access** to the system password file.
- **Deploy** an intrusion detection system to detect compromise and trigger re-issuance of affected passwords.

Specific Account Attack

Restrict the number of consecutive login attempts allowed for any given account to limit brute-force targeting.

Popular Password Attack

- **Enforce policies** that prohibit the use of common or well-known passwords.
- **Scan and correlate authentication requests** by IP address and client cookies to detect distributed attacks.

Guessing Against Single User

- **Require** passwords of sufficient length and complexity (mixed character sets, no well-known identifiers).
- **Mandate periodic password changes** to reduce the exposure window.

Password Vulnerabilities & Countermeasures (II)

Workstation Hijacking

Implement automatic logout after a pre-defined idle period and deploy intrusion detection systems to identify unauthorized session takeover.

Exploiting User Mistakes

Provide user training on secure password practices. Combine password authentication with an additional method (MFA) and monitor for anomalous login behavior.

Multiple Password Reuse

Enforce organizational policies that explicitly discourage or prevent reuse of the same password across multiple systems or services.

Electronic Monitoring

Protect password transmission with strong encryption (e.g., TLS). Avoid transmitting passwords in cleartext over any network, particularly untrusted ones.

Why Password-Based Authentication Remains Popular

Despite its vulnerabilities, [password-based authentication persists](#) because alternatives introduce their own practical barriers.

Four key reasons explain its staying power:

1 Hardware Dependency of Alternatives

Client-side hardware such as fingerprint scanners and smart card readers **requires compatible authentication software deployed on both the client and server sides** — *a significant infrastructure burden.*

2 Cost and Inconvenience of Physical Tokens

Smart cards and hardware tokens are **expensive to issue and inconvenient to carry**, especially when users need multiple tokens for multiple services.

3 Single Point of Failure in SSO

Single sign-on schemes that use non-password techniques introduce a single point of security risk — **if that mechanism is compromised, all services are exposed.**

4 Limitations of Password Managers

Automated password managers have **poor** cross-platform roaming and synchronization support, and their usability **has not been adequately** researched or standardized.

Hashed Passwords & the Role of Salt

- Storing passwords as plain text is never acceptable. Instead, a **one-way cryptographic hash** is stored alongside a random **salt** value.
- Salting** provides three critical security benefits:

No Duplicate Visibility

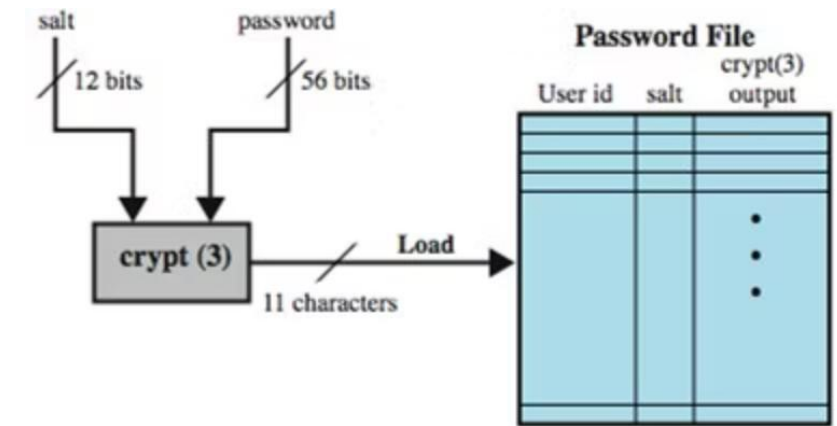
Even if two users choose the same password, the stored hashes will differ due to unique salts.

Harder Offline Attacks

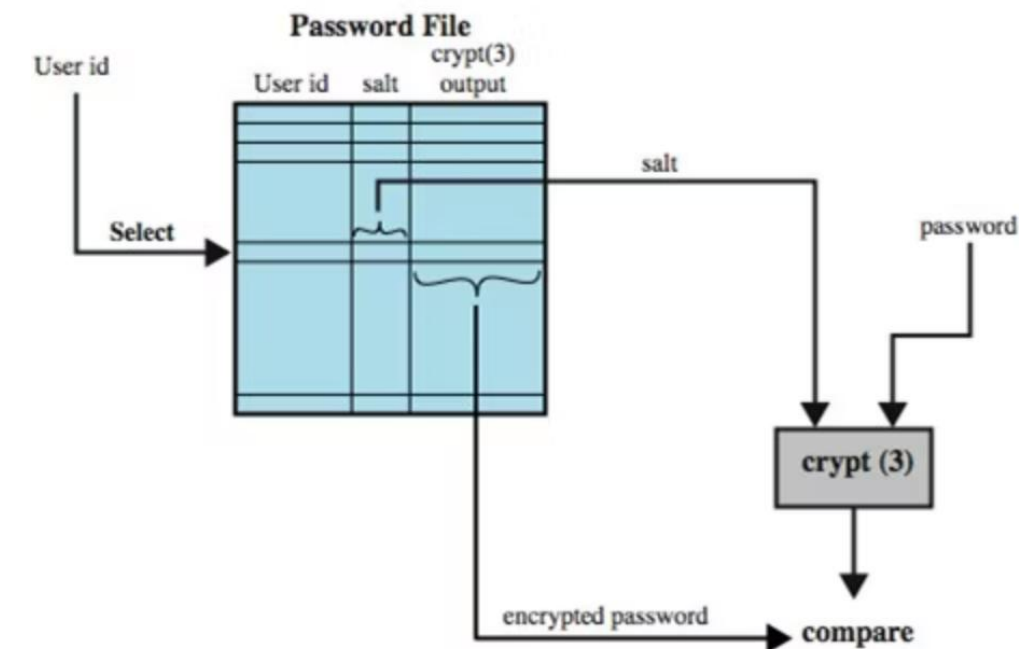
Salting significantly increases the computational cost of offline dictionary attacks.

Cross-System Privacy

An attacker cannot tell if a user reused the same password across multiple systems.



(a) Loading a new password



(b) Verifying a password

Unix Implementation of Password Schemes

Original DES Scheme

8-character password as 56-bit key.
12-bit salt modifies DES into a one-way hash.
Output is an 11-character sequence.
Now considered very insecure.

1

2

MD5 (Modern Linux)

48-bit salt. Unlimited password length.
Password hashed with a 1,000-iteration inner loop.
Produces a 128-bit hash output.

3

Bcrypt (OpenBSD)

Based on the Blowfish block cipher.
Uses a **128-bit salt** to produce a 192-bit hash value.
Tunable cost factor for future-proofing.

- Each successive scheme increases salt length and computational complexity, making offline cracking attacks progressively more expensive.



Password Cracking: Dictionary & Rainbow Table Attacks

Dictionary Attack

The attacker tries each word — and obvious variants — from a large dictionary against the hashes stored in the password file.
Effective against weak, common passwords.

Rainbow Table Attack

Pre-computes tables of hash values for all salt combinations across a large set of candidate passwords.

A **1.4 GB table** can crack **99.9%** of alphanumeric Windows passwords in under 14 seconds.

Mitigation: use **larger salts** and **longer hash lengths** to make precomputation infeasible.

Password Cracking: Exploiting User Behavior

- **Technical attacks** are compounded by predictable human behavior.
- **Attackers** exploit the tendency of users to choose short, simple, or guessable passwords — reducing the practical search space dramatically.

Short Passwords

- Studies show ~**3%** of users chose passwords of 3 characters or fewer.
- Systems should **reject passwords** below a minimum length threshold.

Guessable Passwords

- **Crackers** use curated lists of likely passwords based on common patterns.
- One study of 14,000 encrypted passwords found nearly **1 in 4** could be guessed.

Speed of Attack

On modern hardware, an attacker can compute **all variants of likely passwords in approximately 1 hour** — and only needs a single successful crack to gain entry.

Password Cracking: Modern Approaches

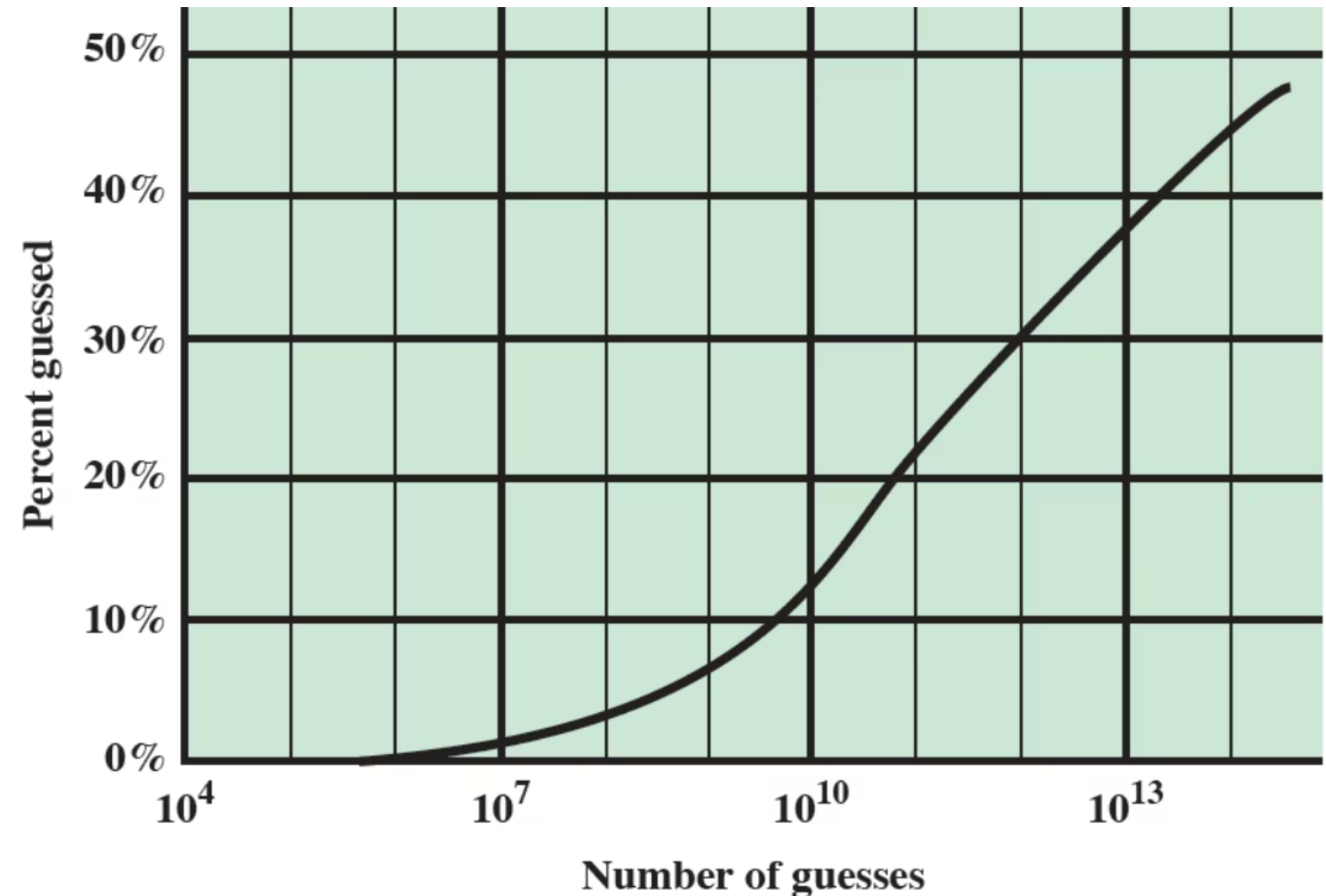
While users now choose better passwords and organizations enforce stronger policies, **password cracking remains highly successful due to two persistent factors:**

Low-Cost Compute Power

Cloud computing and GPU farms make it economically viable to perform billions of hash computations per second, enabling fast offline cracking at scale.

Password Leakage

Frequent large-scale data breaches expose millions of hashed (and sometimes plaintext) passwords, providing attackers with ready-made cracking datasets and real-world frequency analysis.



As the number of guesses increases exponentially (x-axis), the percentage of passwords cracked rises sharply — illustrating the power of modern brute-force approaches.

Password File Access Control

- **Blocking access** to the password file is **a critical first line of defense** against offline guessing attacks.
- Systems should make the password file available **only to privileged users**, often using a separate **shadow password file** accessible only via `su`.

Even with access control in place, residual vulnerabilities remain:

- **Exploitation of operating system bugs** that bypass file permission checks.
- **Accidental misconfiguration of permissions** making the file world-readable.
- Users **reusing the same password** on other less-protected systems.
- **Access via unprotected backup media** that bypasses OS-level controls.
- Sniffing passwords from **unencrypted network traffic** during transmission.

Password File Access Control: Linux Shadow Files

`/etc/passwd`

- World-readable file containing user account information.
- **In older Linux versions**, it also contained hashed passwords — visible to all users and thus vulnerable to rainbow table and password reuse attacks.
- **Today**, it contains *no* password hashes.

`/etc/shadow`

- Introduced to separate hashed passwords from general user information.
- Accessible and modifiable only by the root user.
- Prevents unprivileged users from accessing password hashes, significantly reducing offline attack risk.

- ❏ The separation of `/etc/passwd` and `/etc/shadow` is a foundational access control hardening measure in Linux systems. Ensure permissions on `/etc/shadow` are set to `640` or stricter.

Password Selection Strategies

- The goal of any password selection strategy is to **eliminate guessable passwords** while keeping credentials easy enough for legitimate users to remember.
- **Several techniques** can be deployed at varying levels of enforcement.



User Education

Train users to understand what makes a password weak or strong and why password hygiene matters.



Computer-Generated Passwords

System-generated passwords are highly random but can be hard to remember, affecting adoption.



Reactive Password Checking

Periodically scan existing password files using cracking tools to identify and expire weak passwords.



Proactive Password Checking

Evaluate password strength at the moment of selection, rejecting those that fail complexity or dictionary checks before they are ever stored.

Proactive Password Checking

- Proactive **checking enforces password quality at the time of selection**, preventing weak passwords from ever entering the system.
- **Three complementary mechanisms** are used:

Rule Enforcement

- **Require** 8+ characters combining uppercase, lowercase, numeric, and punctuation characters.
- **Rules** can be tuned via a proactive checker to balance security and usability.

Password Cracker List

- **Maintain** a database of known bad/compromised passwords.
- **Check** each new password submission against this list.
- **The trade-off** is time and storage overhead for large dictionaries.

Bloom Filter

- **Build a compact hash table** from a dictionary of unacceptable passwords using multiple hash functions.
- **Check** the candidate password against the table — fast, space-efficient, with a small false-positive rate.

Bloom Filter

A Bloom filter of order k uses a set of k independent hash functions $H_1(x)$, $H_2(x)$, ..., $H_k(x)$, each mapping a password x into a hash value in the range 0 to $N-1$.

It is an efficient probabilistic structure used to test membership in a set.

1. A hash table of N bits is defined, with all bits initially set to 0.
2. For each password, its k hash values are calculated, and the corresponding bits in the hash table are set to 1. Thus, if $H_i(X_j) = 67$ for some (i, j) , then the sixty-seventh bit of the hash table is set to 1; if the bit already has the value 1, it remains at 1.

- A hash table (an array – actually) of N bits is initialized to 0.
- For each dictionary password, all k hash values are computed and the corresponding bits are set to 1.
- A **false positive** is possible — the filter may incorrectly flag a valid password as weak — but false negatives are not.

$$H_i(X_j) = y \quad 1 \leq i \leq k; \quad 1 \leq j \leq D; \quad 0 \leq y \leq N-1$$

where

X_j = j th word in password dictionary

D = number of words in password dictionary

Token-Based Authentication: Memory Cards

- A **token** is a physical object possessed by the user that provides authentication.
- The simplest form is the **memory card**, which stores but does not process data.

Examples

Magnetic stripe cards (bank cards), electronic memory cards, hotel room key cards, ATM cards (used with a PIN).

Drawbacks of Memory Cards

- **Requires** a special card reader — hardware must be present on both client and server.
- **Loss** or theft of the token creates an immediate security risk.
- Users find them **inconvenient** for computer access (acceptable for ATMs, less so for workstations).

Token-Based Authentication: Smart Cards

Smart cards are far more capable than memory cards — they contain their **own processor, memory (ROM, EEPROM, RAM), and I/O ports**, enabling active cryptographic operations.

Static

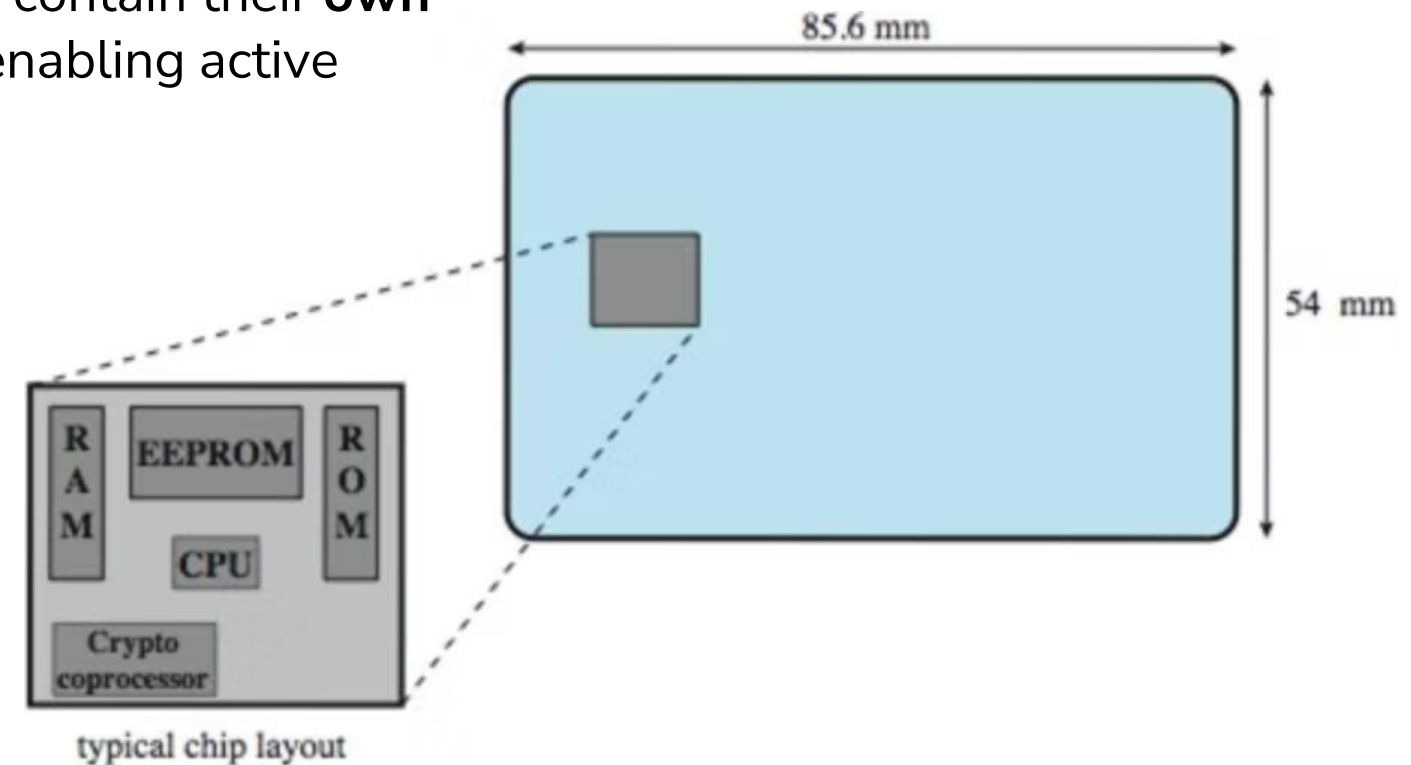
Behaves like a memory card; fixed credentials.

Dynamic OTP

Generates a **new password every minute**, entered manually or electronically.

Challenge-Response

Computer issues a random challenge; the card responds with a cryptographic hash (similar to public key operations).



Standard smart card dimensions (85.6 × 54 mm) with integrated CPU, EEPROM, ROM, RAM, and a crypto coprocessor.

Token-Based Authentication: National eID Cards

An important application of smart card technology is the national **electronic identity (eID)** card, which provides stronger, cryptographically verifiable proof of identity compared to conventional documents.



Stored Data (German eID)

Personal data, document number, Card Access Number (6-digit random), and a machine-readable zone (MRZ).



ePass (Government Use)

Used for border control and official government authentication purposes.



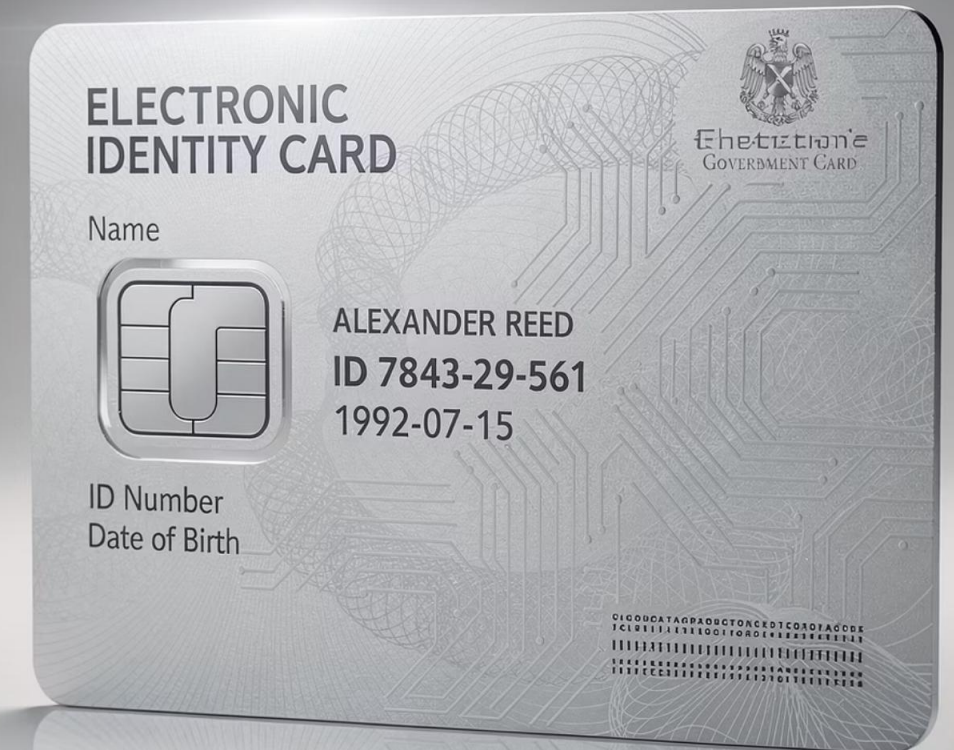
eID (General Use)

Used for general-purpose identity verification in online and in-person services.



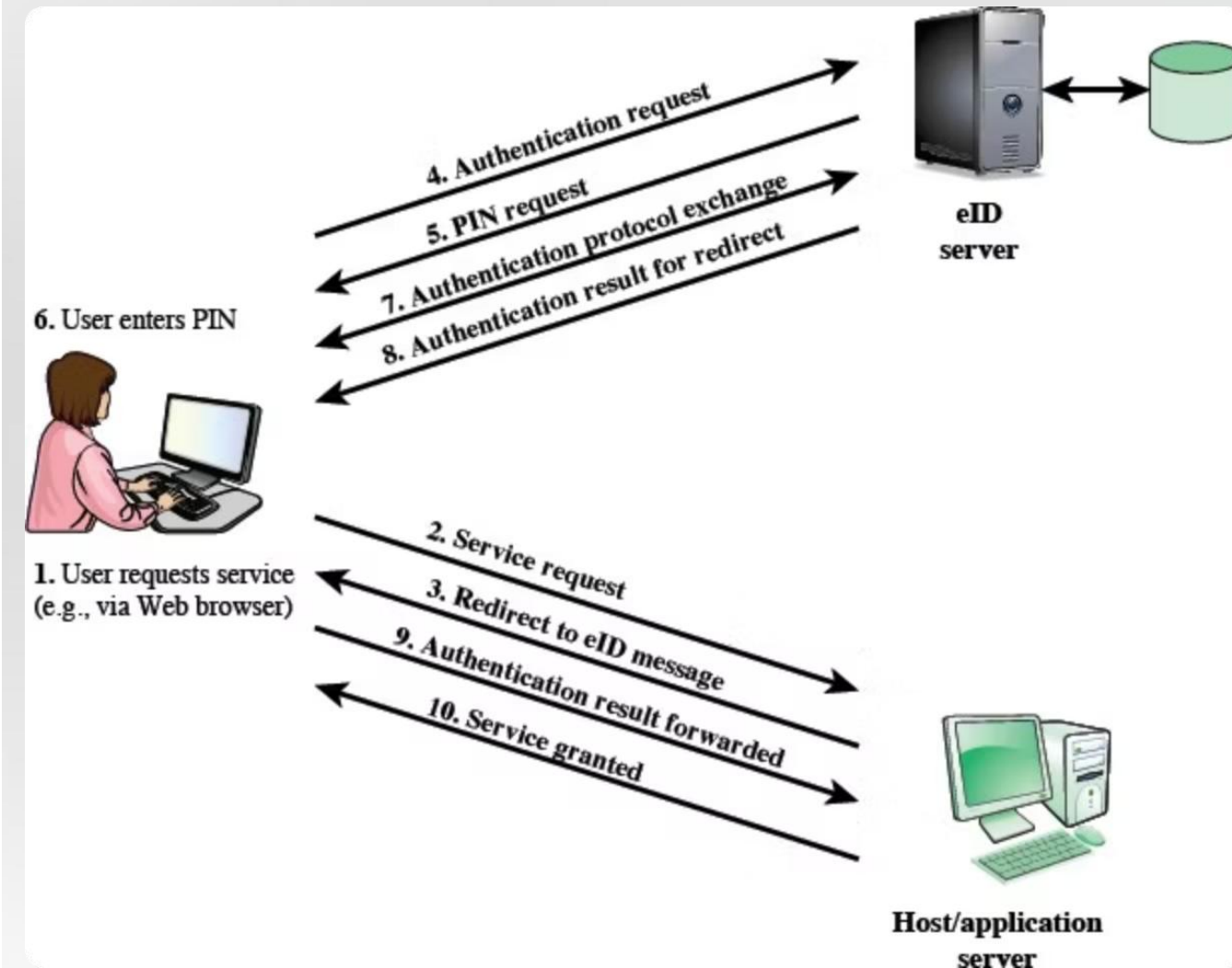
eSign (Digital Signature)

Can store a private key and certificate, enabling legally binding digital signatures.



User Authentication Using eID

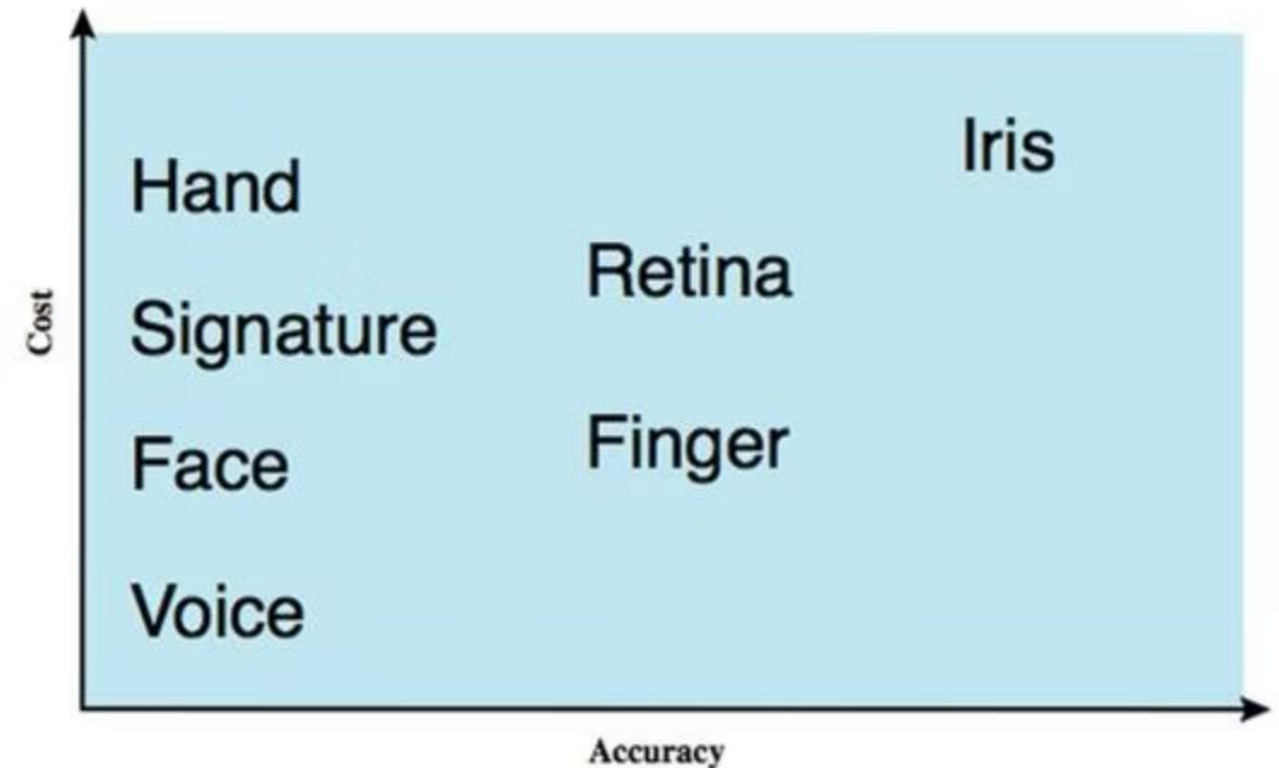
- The eID authentication flow involves a multi-step protocol exchange between the user, the host/application server, and the eID server.
- The user's PIN is entered locally, and the authentication result is forwarded to grant access to the requested service.



Biometric Authentication

Biometric authentication verifies users based on **unique physical or behavioral characteristics**. Different modalities offer different trade-offs between cost and accuracy.

- Facial recognition
- Fingerprint
- Hand geometry
- Retina pattern
- Iris scan
- Signature dynamics
- Voice pattern



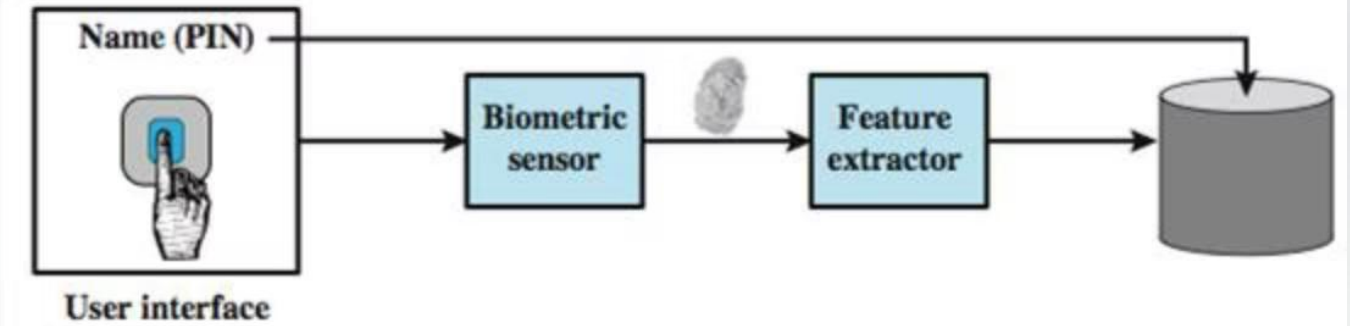
Iris scanning offers the highest accuracy but at greater cost. Voice and face recognition are low-cost but less accurate.

Operation of a Biometric System

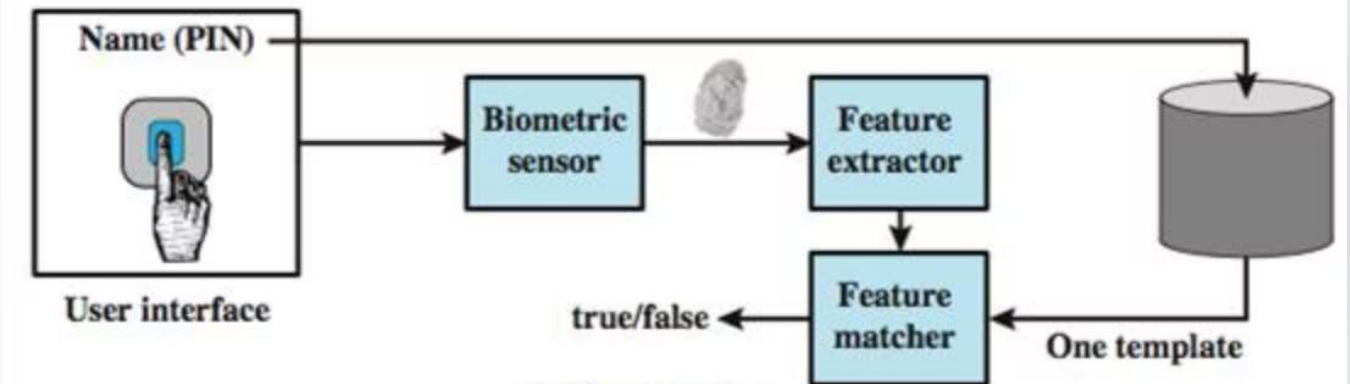
Biometric systems operate in three distinct modes:

- **enrollment** (storing a reference template),
- **verification** (confirming a claimed identity against one template),
- **identification** (searching across N templates to find a match).

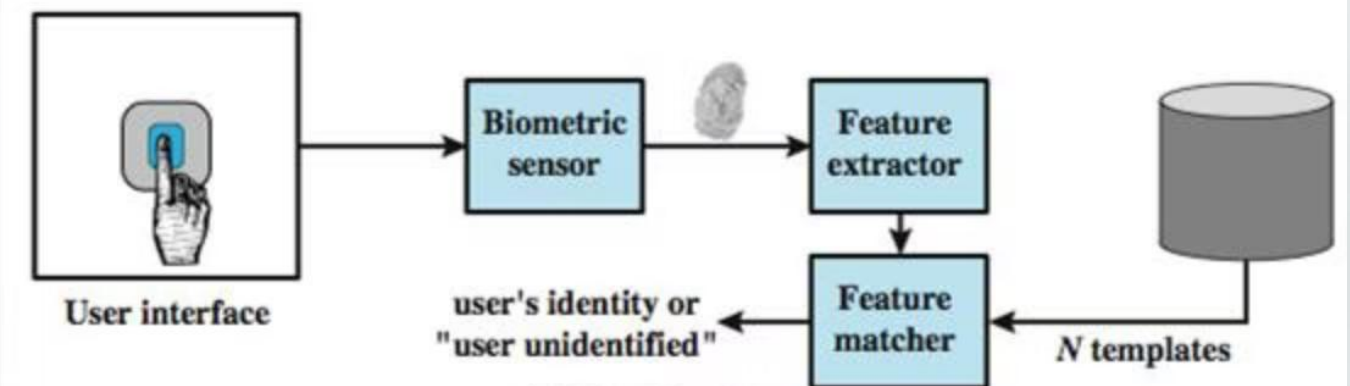
Each mode has different performance and privacy implications.



(a) Enrollment



(b) Verification



(c) Identification

Biometric Accuracy: The Matching Score

- Rather than a binary yes/no, biometric systems produce a **matching score** — a numerical value quantifying the similarity between the live input and the stored template.
- A **decision threshold (t)** determines acceptance or rejection.

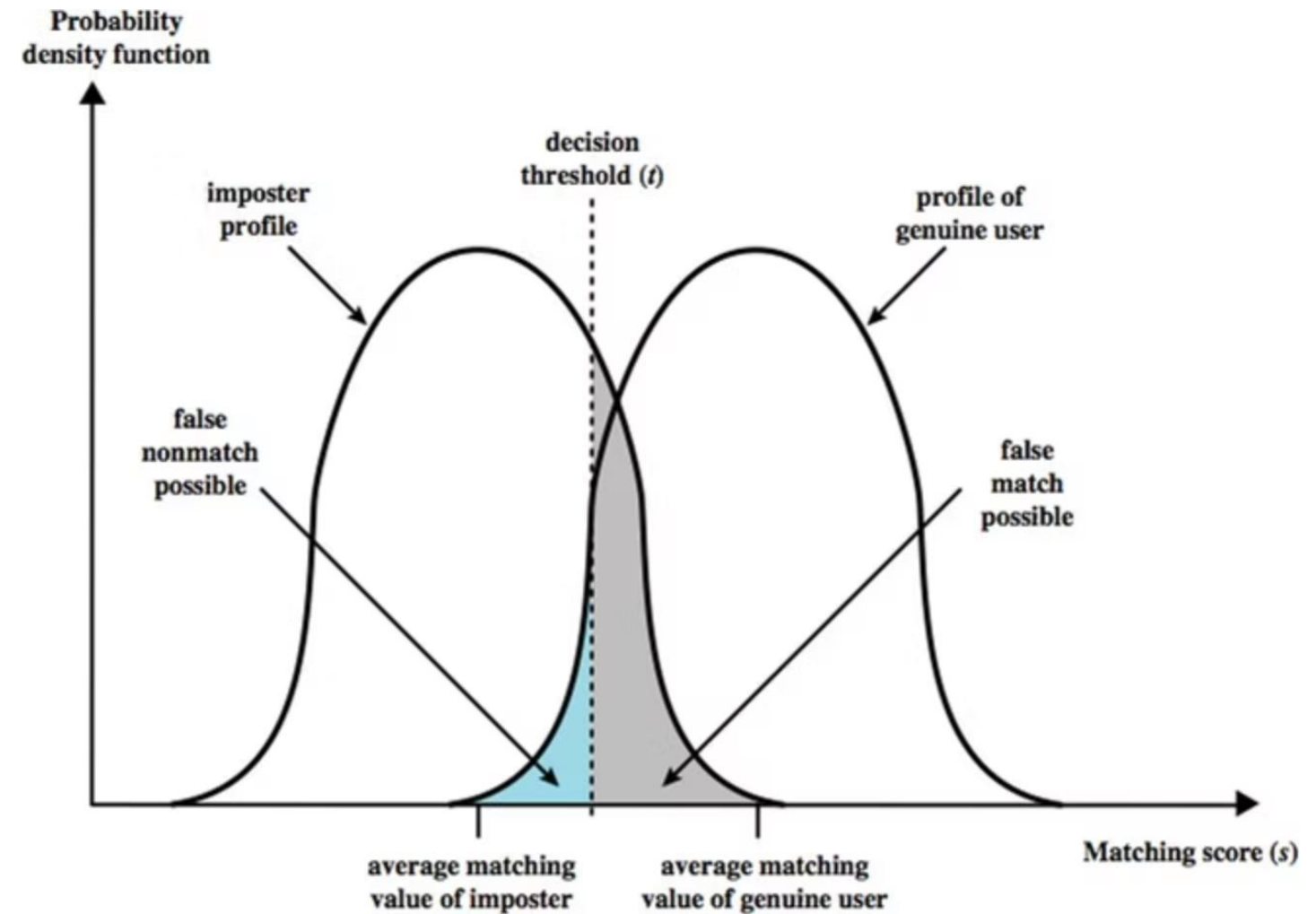
False Match (FMR)

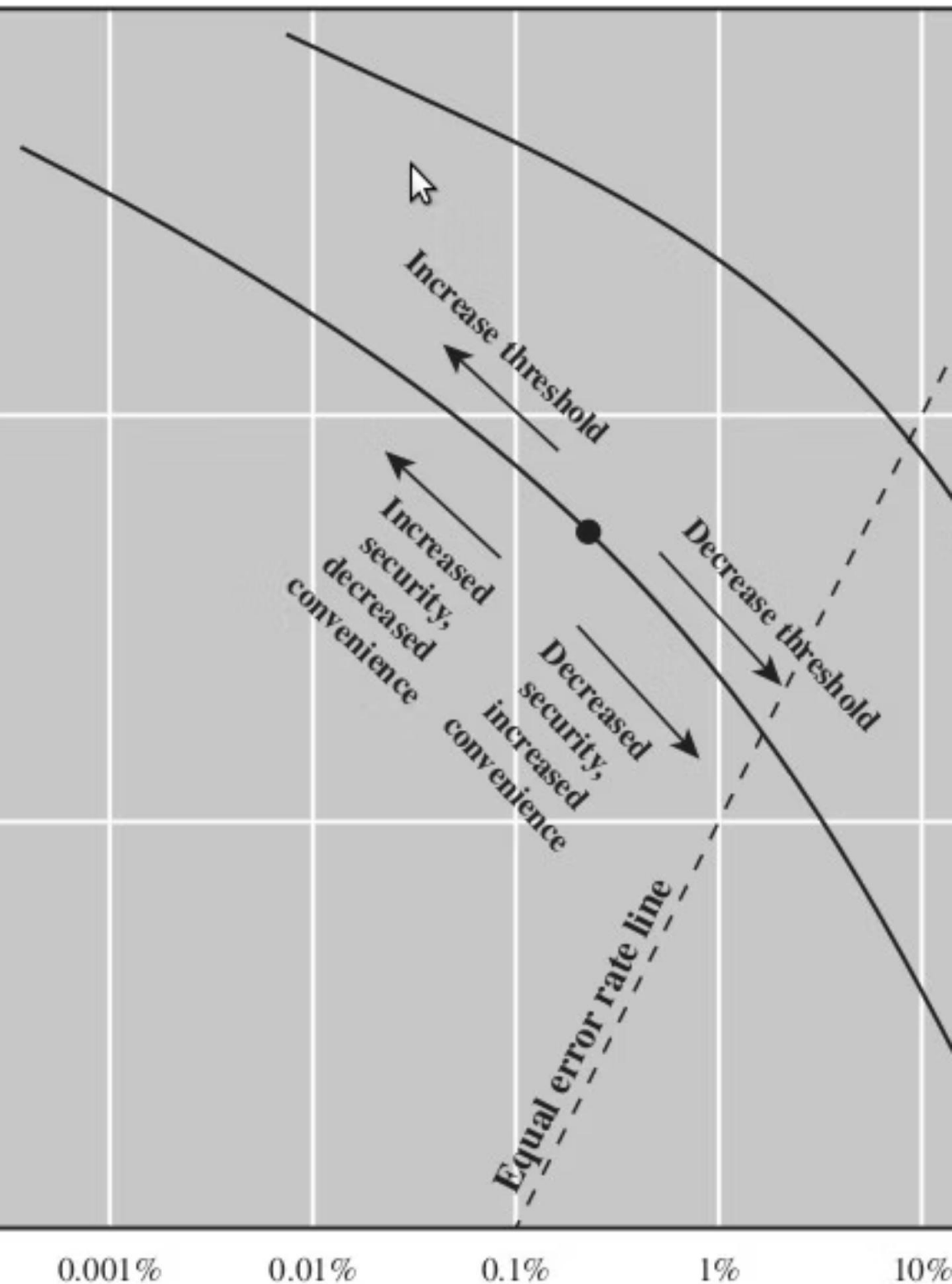
An impostor's score exceeds the threshold — unauthorized access granted.

False Non-Match (FNMR)

A genuine user's score falls below the threshold — legitimate access denied.

Sensor noise and detection inaccuracy contribute to both types of errors. **The threshold** can be tuned to trade off FMR vs. FNMR depending on security requirements.





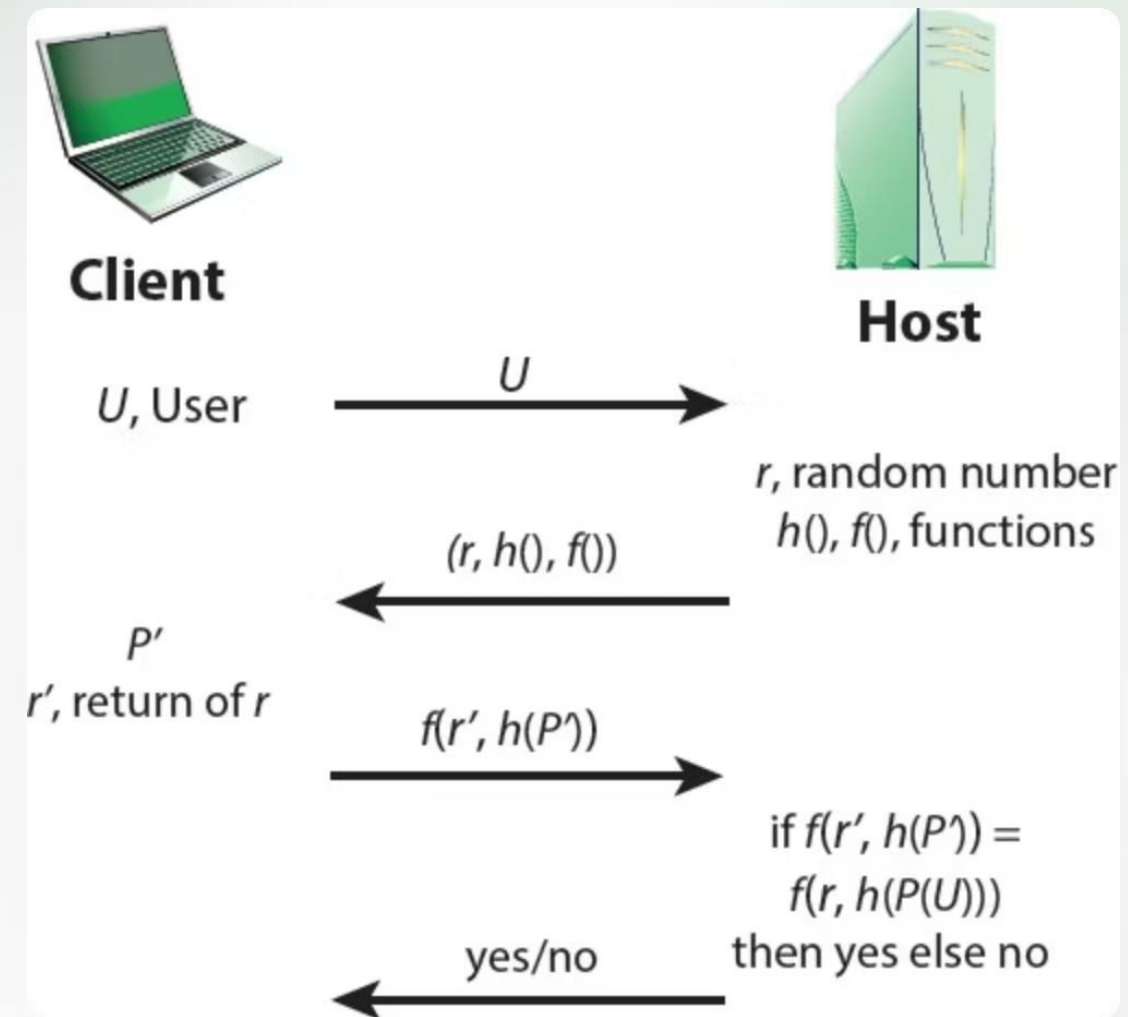
Biometric Accuracy: Reference Table

The following reference table summarizes error rates and performance characteristics across common biometric modalities, helping system designers select the appropriate technology for a given security context.

Remote User Authentication

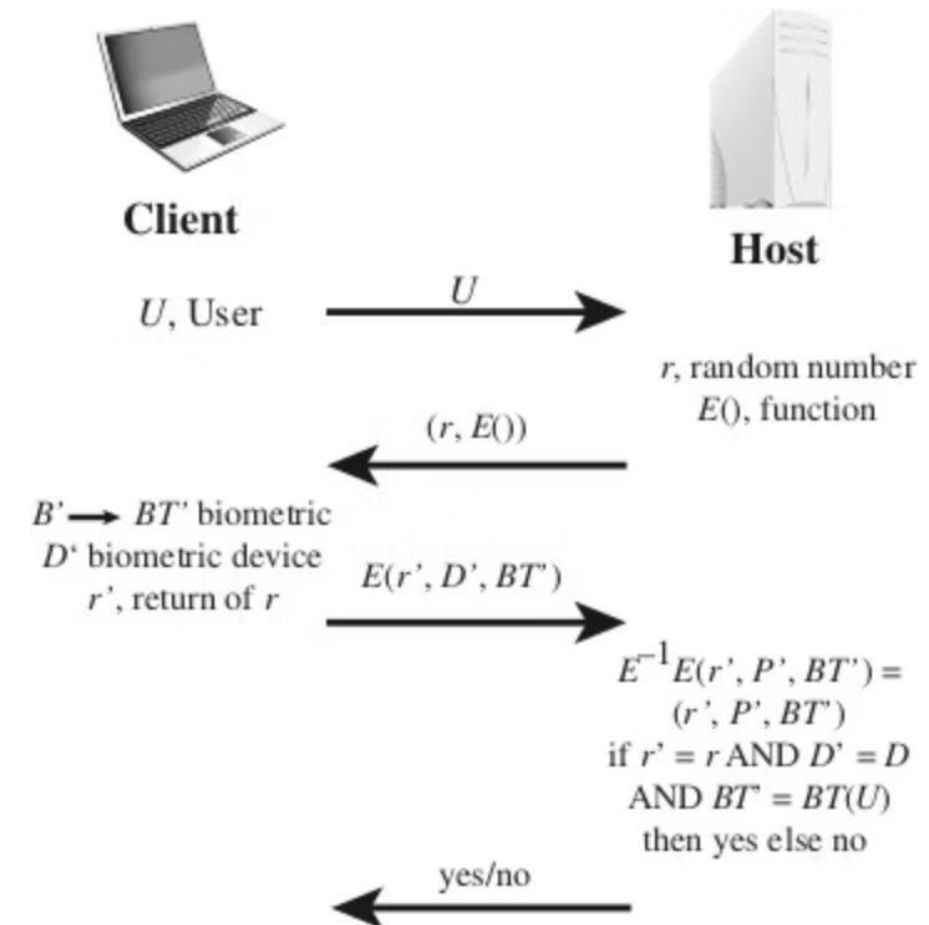
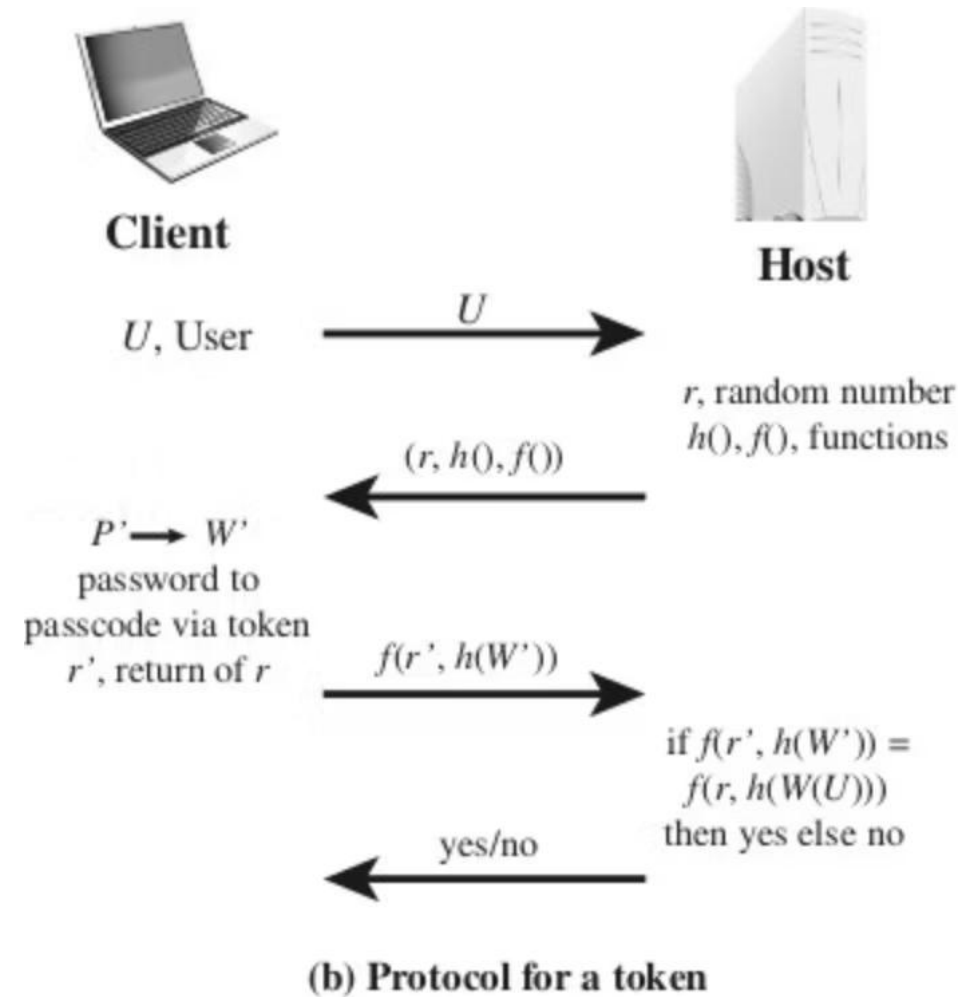
- **Authenticating users over a network** introduces threats that do not exist in local authentication scenarios — most notably **eavesdropping** and **replay attacks**.
- To mitigate these, remote authentication generally relies on **challenge-response protocols** rather than direct password transmission.

□ In a basic challenge-response scheme, the server sends a random nonce to the client. The client combines the nonce with its password/key and returns a hash. This prevents replay attacks because the nonce changes with every session.



Challenge-Response Protocol: Password & Token

These diagrams illustrate the challenge-response protocol applied to both password-based and token-based (symmetric key) remote user authentication.





Client

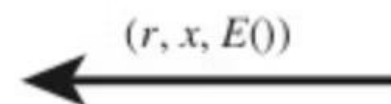
U , User



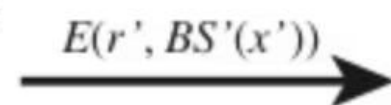
Host



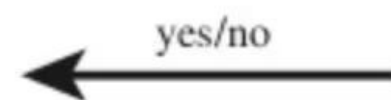
r , random number
 x , random sequence
 challenge
 $E()$, function



$B', x' \rightarrow BS'(x')$
 r' , return of r



$E^{-1}E(r', BS'(x')) =$
 $(r', BS'(x'))$
 extract B'
 from $(r', BS'(x'))$
 if $r' = r$ AND $x' = x$
 AND $B' = B(U)$
 then yes else no



(d) Protocol for dynamic biometric

Challenge-Response Protocol: Public Key

The public-key variant of the challenge-response protocol eliminates the need to share a secret password.

The server encrypts a nonce with the user's public key; only the legitimate user — holding the private key — can decrypt and return the correct response, proving identity without revealing any secret over the network.

Case Study: Dictionary Attacks on SSH

This case study examines real-world dictionary attack attempts targeting SSH login on **port 22**. Understanding the distinction between attack types is essential for effective defense.

```
tail -f /var/log/auth.log | sed G
```

Dictionary Attack

- The attacker first tries a list of probable account names.
- Once a valid account is identified, a large curated set of likely passwords is tried against it.
- The search space is focused and efficient.

Brute Force Attack

- The attacker tries all possible combinations of usernames and passwords.
- Time and memory requirements grow exponentially with the length of credentials — making it slower but exhaustive.

Detecting Dictionary Attacks with Log Scanning (I)

Examining authentication logs (/var/log/auth.log) reveals dictionary attack patterns: **rapid sequential attempts with different account names from the same IP address, followed by password guessing against identified accounts.**

📄 **Try it:** Check the source IP **61.163.228.117** using www.ip2location.com to identify geolocation and ISP information for the attacking host.

Detecting Dictionary Attacks with Log Scanning (II)

- A common attack pattern targets the **root account** directly, since it exists on every Linux/Unix machine.
- The log entries here show repeated failed authentication attempts against root from a single remote IP — a clear indicator of an automated dictionary attack.

```
Apr 10 16:23:39 moonshine sshd[32305]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rh
Apr 10 16:23:41 moonshine sshd[32305]: Failed password for root from 202.99.32.53 port 42732 ssh2

Apr 10 16:23:48 moonshine sshd[32307]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rh
Apr 10 16:23:50 moonshine sshd[32307]: Failed password for root from 202.99.32.53 port 42976 ssh2

Apr 10 16:23:58 moonshine sshd[32309]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rh
Apr 10 16:23:59 moonshine sshd[32309]: Failed password for root from 202.99.32.53 port 43208 ssh2

Apr 10 16:24:06 moonshine sshd[32311]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rh
Apr 10 16:24:08 moonshine sshd[32311]: Failed password for root from 202.99.32.53 port 43439 ssh2

Apr 10 16:24:15 moonshine sshd[32313]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rh
Apr 10 16:24:17 moonshine sshd[32313]: Failed password for root from 202.99.32.53 port 43659 ssh2

Apr 10 16:24:24 moonshine sshd[32315]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rh
Apr 10 16:24:26 moonshine sshd[32315]: Failed password for root from 202.99.32.53 port 43901 ssh2

Apr 10 16:24:33 moonshine sshd[32317]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rh
Apr 10 16:24:35 moonshine sshd[32317]: Failed password for root from 202.99.32.53 port 44128 ssh2

Apr 10 16:24:42 moonshine sshd[32319]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rh
Apr 10 16:24:44 moonshine sshd[32319]: Failed password for root from 202.99.32.53 port 44352 ssh2

Apr 10 16:24:51 moonshine sshd[32321]: pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0 tty=ssh ruser= rh
Apr 10 16:24:53 moonshine sshd[32321]: Failed password for root from 202.99.32.53 port 44577 ssh2
```

Detecting Dictionary Attacks with Log Scanning (III)

SSH logs the message "POSSIBLE BREAK-IN ATTEMPT" when a **reverse DNS lookup on the connecting IP fails** to match the forward lookup — indicating the attacker is using a spoofed or mismatched hostname. **This is a strong indicator of malicious activity.**

```
Apr 10 21:42:45 moonshine sshd[787]: reverse mapping checking \
getaddrinfo for 210-68.colo.sta.blacknight.ie [78.153.210.68] \
failed - POSSIBLE BREAK-IN ATTEMPT!
```

Protecting a Home Network in SSH Connections

One of the simplest and most effective measures to restrict SSH access is to use the `/etc/hosts.allow` and `/etc/hosts.deny` files (TCP Wrappers) to whitelist specific IP addresses.

Precedence Rule

`/etc/hosts.allow` takes precedence over `/etc/hosts.deny`. Entries in `allow` are evaluated first.

IP Whitelisting

Replace `xxx.xxx.xxx.xxx` with the specific IP address from which SSH connections should be permitted. All other sources are denied.

```
/etc/hosts.allow : sshd: xxx.xxx.xxx.xxx  
  
/etc/hosts.deny : ALL: ALL
```

Effect

Ensures that SSH connections from unauthorized IP addresses are blocked before authentication is even attempted.

Thwarting Dictionary Attacks with fail2ban

fail2ban is a widely used, log-scanning-based **intrusion prevention tool**. It **automatically detects brute-force** and dictionary attack patterns and blacklists offending IPs at the firewall level.

Login Attempt Tracking

Monitors the number of failed login attempts within a pre-specified time interval to identify automated attack behavior.

Automatic Blacklisting

Adds iptables firewall rules to block the offending IP address after the threshold is exceeded.

Regex-Based Filtering

Identifies malicious behavior patterns from any application that writes a log file, using configurable regular expression filters.

Application Coverage

Supports a wide range of applications via `/etc/fail2ban/jail.conf`. Custom server applications can also be brought under fail2ban monitoring.



fail2ban: Installation & Configuration

- Installing and enabling fail2ban on a Debian/Ubuntu system is straightforward.
- By default, it monitors `/var/log/auth.log` and can be extended to any application by setting `enabled=true` in the appropriate section of `jail.conf`.

01

Install

```
sudo apt update and sudo apt install fail2ban
```

Status

```
| - Number of jail:      1
' - Jail list:  sshd
```

02

Enable for an Application

Set `enabled=true` in the desired section of `/etc/fail2ban/jail.conf` (or create a `jail.local` override).

```
[sshd]
enabled = true
port    = ssh
filter  = sshd
logpath = /var/log/auth.log
maxretry = 5
bantime = 3600      ← ban for 1 hour

[apache-auth]
enabled = true
port    = http,https
filter  = apache-auth
logpath = /var/log/apache2/error.log
maxretry = 3
bantime = 600
```

03

Verify Status

Use the commands shown below to confirm the service is running and actively monitoring.

Screenshot: /etc/fail2ban/jail.conf

The `jail.conf` file defines fail2ban's behavior for each monitored service — including the ban time, find time window, maximum retries, and the log file to monitor. Each `[section]` corresponds to one monitored application or service.

```
#
# WARNING: heavily refactored in 0.9.0 release. Please review and
#          customize settings for your setup.
#
# Changes:  in most of the cases you should not modify this
#           file, but provide customizations in jail.local file,
#           or separate .conf files under jail.d/ directory, e.g.:
#
# HOW TO ACTIVATE JAILS:
#
# YOU SHOULD NOT MODIFY THIS FILE.
#
# It will probably be overwritten or improved in a distribution update.
#
# Provide customizations in a jail.local file or a jail.d/customisation.local.
# For example to change the default bantime for all jails and to enable the
# ssh-iptables jail the following (uncommented) would appear in the .local file.
# See man 5 jail.conf for details.
#
# [DEFAULT]
# bantime = 1h
#
# [sshd]
# enabled = true
#
# See jail.conf(5) man page for more information

# Comments: use '#' for comment lines and ';' (following a space) for inline commen

[INCLUDES]

#before = paths-distro.conf
before = paths-debian.conf

# The DEFAULT allows a global definition of the options. They can be overridden
# in each jail afterwards.

[DEFAULT]

#
# MISCELLANEOUS OPTIONS
#

"jail.conf" [readonly] 980 lines, 25607 bytes
```

iptables After fail2ban Installation

- fail2ban dynamically inserts rules into iptables to block offending IP addresses.
- The filter table (consulted for incoming packets) consists of three chains: INPUT, FORWARD, and OUTPUT.
- Use sudo iptables -L to inspect the current ruleset.

```
Chain INPUT (policy ACCEPT)
target     prot opt source                destination
f2b-sshd    tcp  --  anywhere              anywhere            multiport dports ssh

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination

Chain f2b-sshd (1 references)
target     prot opt source                destination
RETURN     all  --  anywhere              anywhere
```

```
bantime = 3600

findtime = 3600

maxretry = 5

mta = sendmail

destemail = root@localhost

action = %(action_mwl)s
```

Configuring fail2ban to Monitor Only SSH

- By default, fail2ban may monitor multiple services. To restrict monitoring to SSH only, create or edit `/etc/fail2ban/jail.local` — **not** `jail.conf` (which is overwritten on upgrades).
- The `jail.local` file overrides defaults without modifying the base configuration.

❏ Always make custom changes in `/etc/fail2ban/jail.local`, not in `jail.conf`. The `.local` file takes precedence and will not be overwritten during package updates.

denyhosts: SSH Intrusion Prevention

Although **denyhosts** has been removed from Ubuntu 20.04 and later, it remains widely deployed on older systems and is worth understanding from a security operations perspective.

How It Works

- Monitors SSH authentication attempts on port 22.
- After a configurable number of failures, the offending IP is added to `/etc/hosts.deny` to block further connections.

Synchronization

Downloads shared blacklists from the internet and merges them with local entries in `/etc/hosts.deny`, leveraging community threat intelligence.

Key Files

- Configuration: `/etc/denyhosts.conf`
- Log file: `/var/log/denyhosts`

auth.log After denyhosts Is Activated

- Once denyhosts detects a sufficient number of failed login attempts from an IP, it adds that host to `/etc/hosts.deny`.
- Subsequent connection attempts from that IP are blocked at the TCP Wrappers layer, before SSH even processes the request.

AND FINALLY CAUGHT BY DENYHOSTS:

```
Apr 25 16:29:50 moonshine sshd[31060]: refused connect from ::ffff:190.12.41.50 (::ffff:190.12.41.50)
```

```
DENY_THRESHOLD_INVALID = 5  
DENY_THRESHOLD_VALID   = 10
```

/etc/denyhosts.conf

The denyhosts configuration file allows administrators to tune key parameters: the maximum number of failed attempts before blacklisting, the time window, synchronization settings, and the paths to monitored log files.

The screenshot below shows a typical configuration.

Password Cracking Despite One-Way Hash

- **One-way hash functions** do not make password cracking impossible — they only make it computationally harder.
- For short or limited-character passwords, exhaustive precomputation remains entirely feasible.

The Scale Problem

A 6-character password space contains only $2^{66} = 308,915,776$ possible values. A lookup table of hashes for all such passwords can crack any 6-character password in milliseconds.

The Attacker's Goal

Attackers don't need to crack every password — just one or two accounts where they can install malware or escalate privileges. Cracking short passwords is more than sufficient for this purpose.

- ❏ **Solution:** Combining **salt** (unique random bits per password) with a variable **round count** (iterated hashing) defeats precomputed table attacks and dramatically increases the cost of brute-force attempts.

Entries in /etc/shadow

- Each entry in /etc/shadow consists of **9 colon-separated fields**.
- **The second field** stores the password hash encoded in the **Modular Crypt Function (MCF)** format, which embeds the algorithm identifier, salt, and hash value together.

01

Username

The account login name (must match /etc/passwd).

02

Password Hash (MCF)

Contains algorithm ID, salt, and hash output in a single encoded string.

03

Last Change Date

Days since epoch (Jan 1, 1970) when the password was last changed.

04

Min / Max Age

Minimum days before a password change is allowed; maximum days before a change is forced.

```
identifier: 6
number of rounds: 40000
salt: ZVzZ72hf
actual hash value: Tf19cHUK0g.nf.I/Bpn5jd3jokKMEAIHssRW20EUGfneuTUzkhNmGv9iDhjfeDpJtqOyGjtSeXSq8
```

The Purpose of the Round Value

Password Hashing Scheme	Identifier
md5_crypt	1
bcrypt	2
bcrypt	2a
bcrypt	2x
bcrypt	2y
bsd_nthash	3
sha256_crypt	5
sha512_crypt	6
sun_md5_crypt	md5
sha1_crypt	sha1

Why Iterate the Hash?

- Applying the hash function a variable number of times (**rounds**) multiplies the computational cost of every crack attempt.
- An attacker must perform the same number of iterations per candidate password, making large-scale cracking far more expensive.

Future-Proofing

- As hardware becomes faster and storage cheaper, the round count can be increased to maintain the same level of protection — without changing the underlying algorithm.
- Salt defeats lookup-table attacks; rounds defeat brute-force attacks.

Salt: How It Works

- A **salt** is a randomly chosen bit pattern combined with the user's password *before* hashing.
- It ensures that two users with the same password will have completely different stored hashes, and makes precomputed table attacks infeasible.

Salt in /etc/shadow

The example salt `ZVzZ72hf` is 8 Base64 characters — each encoding 6 bits — yielding a **48-bit salt word**.

How Salt Is Used

In practice, the hashing scheme creates a repetitive concatenation of salt bits and password bits before hashing.

The exact structure depends on the password hashing algorithm in use.

Security Guarantee

Salt is considered strong enough to defeat all lookup-table-based attacks for the foreseeable future, especially when combined with a sufficient round count.

Resources & Acknowledgments

Core Reading

Chapter 3 — Computer Security: Principles and Practice William Stallings and Lawrie Brown

Supplemental Reading

Lecture 24 (Sections 24.1, 24.2, 24.3, 24.5)

engineering.purdue.edu/kak/compsec/

Acknowledgment

Lecture slides covering sections on dictionary attacks and intrusion detection (slides 29–42 in the original deck) are adapted from **Lecture 24** of **Avinash Kak**, Purdue University.

<https://engineering.purdue.edu/kak/compsec/>